CONTEXTUALIZACIÓN COMPLETA DEL PROYECTO ROKOLA Y CONTINUIDAD DEL DESARROLLO

Quiero que actúes como un desarrollador de software senior especializado en Kotlin, Spring Boot, PostgreSQL, arquitectura backend, desarrollo de aplicaciones móviles y sistemas web.

Necesito que me acompañes paso a paso en el desarrollo de mi proyecto universitario llamado **Rokola**. Antes de proponer cambios o escribir código, lee cuidadosamente todo el contexto para comprender qué hemos hecho, qué decisiones hemos tomado, qué problemas tenemos y qué falta por desarrollar.

Tu objetivo es continuar el proyecto desde su estado actual, evitando repetir trabajo, cambiar decisiones sin justificación o introducir complejidad innecesaria.

---

1. DESCRIPCIÓN GENERAL DEL PROYECTO

Rokola es un sistema móvil-web de solicitudes musicales, reproducción y analítica para establecimientos nocturnos, como bares, tabernas y otros establecimientos de esparcimiento.

El propósito es permitir que los clientes participen en la selección de música mediante sus teléfonos celulares, mientras que el personal del establecimiento administra las solicitudes y controla la reproducción.

Actualmente, en muchos establecimientos, la selección musical depende exclusivamente de un DJ, administrador, mesero o bartender. Los clientes realizan solicitudes informales que pueden perderse, olvidarse o no tener prioridad.

Rokola busca solucionar este problema mediante un sistema de participación musical.

Flujo general del sistema

1. El cliente escanea un código QR del establecimiento.
2. Accede a una interfaz móvil sin instalar una aplicación adicional.
3. El sistema identifica su dispositivo sin solicitar datos personales innecesarios.
4. El cliente busca canciones o videos musicales.

5. Puede solicitar canciones y votar por solicitudes existentes.
6. Si otro cliente solicita una canción que ya está pendiente, se incrementa su popularidad mediante un voto adicional.
7. El sistema organiza la cola según los votos y la antigüedad de las solicitudes.
8. La reproducción se realiza en una pantalla o Smart TV mediante contenido de YouTube.
9. El personal del establecimiento administra la sesión, las solicitudes y la reproducción.
10. El sistema registra información para generar estadísticas y analítica.

El proyecto tiene un alcance académico y un producto mínimo viable (PMV). No se pretende desarrollar desde el principio una plataforma comercial completa ni un sistema profesional de sonido.

---

2. ARQUITECTURA Y TECNOLOGÍAS ELEGIDAS

Hemos decidido trabajar con una arquitectura de backend monolítica modular.

Las tecnologías seleccionadas son:

- **Lenguaje backend:** Kotlin.
- **Framework:** Spring Boot.
- **Java:** versión 21.
- **Gestor de dependencias:** Gradle con Kotlin DSL.
- **Base de datos:** PostgreSQL.
- **Migraciones:** Flyway.
- **Control de versiones:** Git y GitHub.
- **Despliegue de PostgreSQL:** Railway.

La arquitectura prevista contempla:

Aplicación móvil

Desarrollada con Kotlin para Android. Sus funcionalidades incluyen:

- Escaneo de códigos QR.
- Identificación del dispositivo.
- Búsqueda de videos musicales.
- Solicitud de canciones.
- Votación.
- Consulta del estado de las solicitudes.

Página web pública

- Visualización de la cola musical.
- Reproducción de videos de YouTube.
- Actualización de la información en tiempo real.

Página web administrativa

- Gestión de establecimientos.
- Administración de usuarios y roles.
- Configuración del servicio musical.
- Administración de solicitudes.
- Control de reproducción.
- Consulta de estadísticas.

Backend

Se encarga de centralizar la lógica del negocio, la comunicación con PostgreSQL, la integración con YouTube y la exposición de servicios REST.

Se contempla el uso de WebSocket para funcionalidades que requieran actualización en tiempo real.

---

3. DISEÑO DE LA BASE DE DATOS

Se diseñó una base de datos relacional en PostgreSQL con aproximadamente 21 tablas.

Las tablas principales son:

Tabla	Responsabilidad
establecimiento	Información de cada establecimiento
horario_operacion	Horarios de funcionamiento
configuracion_servicio	Parámetros del servicio musical
codigo_qr	Códigos QR de acceso
rol	Roles y permisos
usuario	Usuarios administrativos
evento_calendario	Eventos especiales
sesion	Sesiones musicales del establecimiento
dispositivo	Identificación de dispositivos clientes

cancion	Información de canciones y videos
solicitud	Solicitudes musicales
voto	Votos de los clientes
bloqueo_contenido	Restricciones de contenido musical
lista_defecto	Listas musicales predeterminadas
elemento_lista	Elementos de las listas
reproduccion	Historial y control de reproducción
busqueda_cache	Caché de búsquedas musicales
evento_analitico	Registro de eventos para estadísticas
accion_admin	Auditoría de acciones administrativas
error_integracion	Registro de errores de servicios externos
ejecucion_pipeline	Registro de procesos de analítica

Decisiones importantes del modelo

Durante el diseño se tomaron las siguientes decisiones:

- No almacenar `votos_total` directamente en `solicitud`, porque el total puede calcularse desde la tabla `voto`.
- Evitar relaciones redundantes entre las tablas.
- Mantener la relación de `voto` con `solicitud` y `dispositivo`, sin agregar información innecesaria de sesión si puede obtenerse mediante la solicitud.
- Diferenciar los roles de administrador y operador.
- Permitir configurar límites de solicitudes y reglas de participación.
- Mantener el historial de reproducción y las acciones administrativas.
- Permitir que el sistema utilice listas predeterminadas cuando no existan solicitudes pendientes.

Reglas principales del negocio

1. Un código QR debe dirigir al establecimiento correspondiente.
2. Un dispositivo debe poder identificarse durante la noche sin solicitar nombre, correo o teléfono.
3. Las sesiones deben tener estados y horarios de funcionamiento.
4. Una solicitud nueva comienza en estado de espera y recibe un voto inicial.

5. Si ya existe una solicitud pendiente de la misma canción, se agrega un voto en lugar de duplicar la solicitud.
6. Un dispositivo no puede votar repetidamente por la misma solicitud.
7. La cola se organiza por cantidad de votos descendente y antigüedad ascendente.
8. Deben existir filtros y bloqueos de contenido.
9. Debe controlarse la repetición de canciones durante un intervalo configurable.
10. Cuando no existan solicitudes, puede utilizarse una lista musical predeterminada.
11. Las acciones administrativas importantes deben quedar registradas.

El diseño de la base de datos se ha trabajado, pero debe verificarse su correspondencia exacta con el esquema que actualmente existe en Railway antes de continuar.

---

4. ESTADO ACTUAL DEL BACKEND

El proyecto Spring Boot se generó mediante Spring Initializr.

La carpeta actual del proyecto es:

`web-musica/rokolaBackend`

El paquete principal utilizado es:

`com.proyectoN.rokolaBackend`

La clase principal es:

`RokolaBackendApplication.kt`

Se ha trabajado con Gradle Kotlin DSL y Java 21.

Estructura modular prevista

```
None
rokolaBackend/
├── src/
│   └── main/
│       ├── kotlin/
│       │   └── com/proyectoN/rokolaBackend/
│       │       ├── RokolaBackendApplication.kt
│       │       ├── config/
│       │       ├── establecimiento/
│       │       ├── usuario/
```

```
│       │       ├── sesion/
│       │       ├── dispositivo/
│       │       ├── cancion/
│       │       ├── solicitud/
│       │       ├── voto/
│       │       ├── reproduccion/
│       │       ├── playlist/
│       │       ├── youtube/
│       │       ├── analitica/
│       │       ├── auditoria/
│       │       └── seguridad/
│       └── resources/
│           ├── application.properties
│           └── db/
│               └── migration/
├── .env
├── .env.example
├── .gitignore
├── build.gradle.kts
└── settings.gradle.kts
```

Esta es la estructura prevista, no necesariamente todos los directorios ya están creados.

Dentro de cada módulo se organizarán las clases según se necesiten: entidades, repositorios, servicios, controladores, DTO y demás componentes.

No quiero que se creen todos los módulos de golpe. Prefiero desarrollar y probar cada funcionalidad progresivamente.

---

5. CONFIGURACIÓN DE CONEXIÓN Y CREDENCIALES

Se decidió utilizar variables de entorno para evitar publicar las credenciales de PostgreSQL.

En `application.properties` se ha trabajado con una configuración similar a esta:

```
None
spring.application.name=rokolaBackend

spring.config.import=optional:file:.env[.properties]

spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

El archivo `.env`, que no debe subirse a GitHub, contiene las credenciales reales:

```
None
DB_URL=jdbc:postgresql://HOST:PUERTO/NOMBRE_BD
DB_USERNAME=USUARIO
DB_PASSWORD=CONTRASEÑA
```

El archivo `.env.example` contiene valores de ejemplo, sin credenciales reales.

En `.gitignore` se incluyó:

```
None
.env
.env.*
!.env.example
.gradle/
build/
out/
```

Para Railway se deben configurar las variables correspondientes directamente en el servicio del backend.

Nunca solicites que comparta contraseñas, tokens o credenciales privadas en el chat.

---

6. CONEXIÓN CON RAILWAY Y PROBLEMAS ENCONTRADOS

Se ha intentado conectar el backend Spring Boot con una instancia PostgreSQL alojada en Railway.

Durante el proceso se encontraron dos errores.

Primer error: autenticación PostgreSQL

Se presentó:

```
None
SQL State : 28P01
FATAL: la autentificación password falló para el usuario "postgres"
```

Se explicó que el código 28P01 corresponde a un problema de autenticación.

También se revisó que la URL JDBC debe tener esta estructura:

```
None
DB_URL=jdbc:postgresql://HOST:PUERTO/NOMBRE_BD
```

Se indicó verificar las variables de Railway:

- PGHOST
- PGPORT
- PGDATABASE
- PGUSER
- PGPASSWORD

También se explicó la diferencia entre conexión pública y conexión privada de Railway.

Segundo error: Flyway

Después de los cambios, el error de autenticación dejó de ser el problema principal y apareció un nuevo error durante el arranque:

```
None
Found non-empty schema(s) "public"
```

```
but no schema history table.
Use baseline() or set baselineOnMigrate to true
```

El stack trace muestra que el error ocurre durante la inicialización de Flyway, antes de que se complete la creación del contexto de Spring Boot.

Esto significa que Flyway se conecta y encuentra un esquema `public` que contiene objetos, pero no encuentra la tabla `flyway_schema_history`.

Se propuso esta configuración:

```
None
spring.flyway.enabled=true
spring.flyway.baseline-on-migrate=true
spring.flyway.baseline-version=1
```

Sin embargo, **todavía no se ha confirmado si esta configuración fue agregada ni si solucionó el problema**.

Tampoco se ha confirmado si las tablas de Railway están completas, parcialmente creadas o si la base de datos está vacía.

Por lo tanto, no debes asumir que el problema ya está resuelto.

Consideraciones importantes sobre Flyway

Antes de modificar la configuración o ejecutar comandos SQL:

1. Averigua si las tablas existentes fueron creadas manualmente.
2. Verifica si el esquema está completo.
3. Comprueba si hay datos que deban conservarse.
4. Determina si corresponde utilizar `baseline`.
5. Evita eliminar tablas o ejecutar operaciones destructivas.
6. Explica cómo deben organizarse las migraciones SQL.
7. Ten presente que, si se establece la versión inicial como 1, las migraciones posteriores deberán tener versiones superiores.

No ejecutes ni recomiendes `DROP SCHEMA`, `DROP TABLE` o reinicializaciones destructivas sin una justificación y confirmación explícita.

---

7. GIT Y GITHUB

El proyecto se encuentra dentro del repositorio `web-musica`.

Se ha trabajado en la rama:

`backend`

Anteriormente el proyecto estaba en una carpeta llamada `backend`, pero se decidió cambiarla a:

`rokolaBackend`

Este cambio fue intencional para reorganizar el proyecto.

Se han revisado comandos como:

```
None
git branch
git status
git add -A
git commit
git push origin backend
```

También aparecieron advertencias de conversión CRLF/LF en archivos de Gradle, que no necesariamente representan errores.

No asumas que todos los cambios ya fueron enviados a GitHub. Si es necesario, pide verificar el estado con `git status`.

---

8. QUÉ FALTA POR HACER

Quiero que organices el desarrollo pendiente por etapas.

ETAPA 1. Terminar la conexión con PostgreSQL

- Confirmar las credenciales y variables de Railway sin exponerlas.
- Verificar el nombre de la base de datos y el esquema.
- Comprobar qué tablas existen.
- Resolver correctamente el problema de Flyway.
- Verificar que Spring Boot inicie sin errores.

- Ejecutar una consulta sencilla para comprobar la conexión.

ETAPA 2. Configurar las migraciones

- Organizar las migraciones SQL en `src/main/resources/db/migration/`.
- Definir una estrategia para el esquema existente.
- Evitar duplicar tablas.
- Verificar que Flyway pueda gestionar cambios futuros.

ETAPA 3. Crear las entidades Kotlin

Representar las tablas de PostgreSQL mediante entidades Kotlin.

Empezar por los módulos fundamentales:

- Establecimiento.
- Usuario y rol.
- Sesión.
- Dispositivo.
- Canción.
- Solicitud.
- Voto.

Después continuar con reproducción, listas musicales, auditoría y analítica.

Las entidades deben corresponder al esquema real de PostgreSQL. No inventes columnas ni relaciones sin justificarlo.

ETAPA 4. Crear repositorios

Implementar el acceso a datos mediante Spring Data JPA o la alternativa que esté justificada.

Verificar las relaciones y consultas necesarias.

ETAPA 5. Implementar servicios

Desarrollar progresivamente las reglas del negocio:

- Validación de sesiones.
- Identificación de dispositivos.
- Búsqueda de canciones.
- Creación de solicitudes.
- Detección de solicitudes duplicadas.
- Registro de votos.
- Restricciones de votación.

- Ordenamiento de la cola.
- Bloqueos de contenido.
- Control de reproducción.

ETAPA 6. Crear controladores REST

Exponer endpoints para que la aplicación móvil y las páginas web puedan comunicarse con el backend.

Definir correctamente:

- Rutas.
- Métodos HTTP.
- DTO.
- Validaciones.
- Respuestas.
- Códigos de estado.
- Manejo de errores.

ETAPA 7. Seguridad

Implementar autenticación y autorización para el personal administrativo.

Diferenciar permisos de administrador y operador.

Mantener el acceso público de los clientes mediante el QR y los mecanismos de identificación del dispositivo, según el diseño del proyecto.

ETAPA 8. Integraciones

- Integración con búsqueda de videos musicales de YouTube.
- Gestión de caché.
- Reproducción de contenido.
- Comunicación en tiempo real mediante WebSocket.

Tener presente que la integración con YouTube debe respetar sus políticas y limitaciones técnicas.

ETAPA 9. Analítica

- Registro de eventos.
- Estadísticas de solicitudes.
- Popularidad musical.
- Participación de dispositivos.
- Historial de reproducción.
- Dashboard administrativo.

ETAPA 10. Pruebas y despliegue

- Pruebas unitarias.
- Pruebas de integración.
- Pruebas de endpoints.
- Pruebas de reglas de negocio.
- Verificación de conexión a Railway.
- Despliegue del backend.
- Integración con el frontend y la aplicación móvil.

---

9. FORMA EN LA QUE QUIERO QUE ME ACOMPAÑES

Es muy importante que sigas estas instrucciones durante todo el desarrollo:

1. **Explícame paso a paso.** Soy estudiante universitario y estoy aprendiendo a desarrollar este tipo de sistemas.
2. No supongas que tengo experiencia avanzada con Kotlin, Spring Boot, Gradle o Flyway.
3. Cuando propongas modificar un archivo, dime su ruta exacta.
4. Cuando proporciones código, entrégalo completo y listo para copiar cuando sea posible.
5. Explica brevemente qué hace cada cambio y por qué es necesario.
6. No me des demasiadas tareas simultáneamente. Prefiero completar una etapa, probarla y después continuar.
7. No cambies la arquitectura o las tecnologías seleccionadas sin explicar claramente por qué sería necesario.
8. No inventes nombres de tablas, columnas, variables o archivos que no hayamos definido.
9. Antes de modificar algo que ya funciona, revisa el contexto y solicita el código o archivo actual si no lo tienes.
10. Si aparece un error, analiza primero el mensaje completo y determina su causa antes de recomendar cambios.
11. Diferencia los errores de código, configuración, conexión, dependencias y base de datos.
12. Prioriza soluciones sencillas y adecuadas para un proyecto universitario.
13. No recomiendes herramientas, servidores o servicios adicionales si no son necesarios.
14. Nunca des por hecho que una solución funcionó sin que hayamos comprobado el resultado.
15. Conserva las decisiones de diseño que ya hemos tomado, salvo que exista una razón técnica para cambiarlas.

---

10. PUNTO EXACTO DESDE DONDE DEBES CONTINUAR

Actualmente estamos intentando conseguir que el backend Kotlin + Spring Boot se conecte correctamente a PostgreSQL en Railway.

La conexión ya avanzó respecto al error inicial de autenticación y ahora el principal problema visible es Flyway:

```
None
Found non-empty schema(s) "public"
but no schema history table.
```

Todavía debemos confirmar el estado de la base de datos y decidir la estrategia correcta de inicialización de Flyway.

No empieces creando entidades ni controladores todavía. Primero ayúdame a resolver correctamente la conexión y las migraciones.

Comienza respondiendo con:

1. Un breve resumen de lo que entendiste del proyecto.
2. El problema técnico actual y su posible causa.
3. Qué información necesitas verificar para resolverlo de forma segura.
4. El primer paso concreto que debo realizar.

Recuerda: quiero desarrollar Rokola de forma ordenada, funcional y progresiva, comprendiendo lo que hacemos en cada etapa.

ACTUALIZACIÓN DEL ESTADO DEL PROYECTO – 05 DE NOVIEMBRE DE 2026

Esta sección actualiza el contexto anterior y tiene prioridad sobre cualquier información contradictoria de las secciones anteriores.

1. Conexión con PostgreSQL en Railway: avance confirmado

Se consiguió que Hibernate estableciera comunicación con la base de datos PostgreSQL alojada en Railway.

En la consola del backend aparece la siguiente información:

```
None
Database JDBC URL
[jdbc:postgresql://gondola.proxy.rlwy.net:19352/railway]
Database driver: PostgreSQL JDBC Driver
Database dialect: PostgreSQLDialect
Database version: 18.6
Default catalog/schema: railway/public
Isolation level: READ_COMMITTED
```

Esto demuestra que Hibernate logró conectarse al servidor PostgreSQL, reconocer su controlador, identificar el dialecto y consultar información del motor de base de datos.

Por lo tanto, la conexión de Hibernate con Railway está establecida.

Sin embargo, todavía falta verificar que el endpoint de prueba `/prueba-bd` ejecute una consulta SQL correctamente.

2. Error anterior de Flyway

Anteriormente se presentó:

```
None
Found non-empty schema(s) "public"
but no schema history table.
Use baseline() or set baselineOnMigrate to true
```

La causa indicada por Flyway es que encuentra objetos en el esquema `public`, pero no encuentra la tabla de historial `flyway_schema_history`.

Se propuso:

```
None
spring.flyway.enabled=true
```

```
spring.flyway.baseline-on-migrate=true
spring.flyway.baseline-version=1
```

Todavía debe verificarse si esta configuración fue agregada y si el error desapareció.

No asumir que Flyway está funcionando correctamente solo porque Hibernate ahora muestra información de conexión.

Antes de continuar, verificar el estado del esquema y evitar modificaciones destructivas.

3. Spring Security: formulario de inicio de sesión

Al ingresar a:

```
None
http://localhost:8080/login
```

apareció un formulario solicitando usuario y contraseña.

En la consola se encontró:

```
None
Using generated security password: ...
```

Se identificó que Spring Security está activando su autenticación predeterminada y generando una contraseña temporal.

Es importante diferenciar:

- Las credenciales del archivo `.env` se utilizan para conectarse a PostgreSQL.
- Las credenciales de `/login` corresponden a la autenticación de Spring Security.
- No son las mismas credenciales.

Todavía no se ha definido ni implementado el sistema definitivo de autenticación de Rokola.

Más adelante se deberá implementar la autenticación de usuarios administrativos y los permisos de administrador y operador.

No desactivar Spring Security ni modificar su configuración sin revisar primero las dependencias y el código actual.

4. Estado actualizado

Componente	Estado
Proyecto Spring Boot con Kotlin	Creado
Gradle Kotlin DSL	Configurado
Java 21	Configurado
PostgreSQL en Railway	Conexión de Hibernate confirmada
Controlador PostgreSQL	Reconocido
Endpoint `/prueba-bd`	Pendiente de verificación
Flyway	Pendiente de verificar
Spring Security	Autenticación predeterminada identificada
Entidades Kotlin	Pendientes
Repositorios	Pendientes
Servicios	Pendientes
Controladores REST funcionales	Pendientes
Autenticación propia de Rokola	Pendiente